

Paper Review: Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement

Vedang Kashyap

*Department of Electronics Engineering (VLSI Design)
Vellore Institute of Technology, Vellore, India*

I. INTRODUCTION

Existing replacement policies, whether recency-based, frequency-based, or hybrid, are each heuristics geared toward a specific class of access pattern, so no single one exploits short-term, medium-term, and long-term reuse simultaneously the way Belady's optimal algorithm does, even though Belady's algorithm itself is infeasible since it requires knowledge of future accesses.

A. Contribution

- Introduces Hawkeye, which reconstructs Belady's optimal solution for past cache accesses and trains a per-PC predictor on it to guide future replacement decisions.
- Introduces OPTgen, an algorithm that efficiently computes what Belady's algorithm would have cached for a given history using liveness intervals, and combines it with Set Dueling to sample only 64 cache sets, keeping total hardware to 28KB for a 2MB LLC.

II. PROPOSAL

Hawkeye treats replacement as binary classification: is an incoming line cache-friendly or cache-averse? OPTgen answers this for past accesses by tracking each line's usage interval, the time between one access and its next reuse, and an occupancy vector recording how many liveness intervals overlap at each point in time; a line is a hit under OPT if the vector never reaches cache capacity across its usage interval, a miss otherwise. Set Dueling limits OPTgen to 64 sampled sets and a Sampled Cache history 8x the cache size, reconstructing OPT with 99% accuracy at low hardware cost. The Hawkeye Predictor is an 8K-entry, PC-indexed table of 3-bit counters trained by OPTgen's verdicts: the PC that last accessed a line is trained positively on an OPT hit, negatively on an OPT miss. On every access, the predictor's binary output sets a 3-bit RRIP value, cache-friendly lines get RRIP 0 and age normally, cache-averse lines get RRIP 7 and are evicted first; if none are cache-averse, the oldest cache-friendly line is evicted as in LRU, and its PC is detrained if present in the sampler.

III. KEY RESULTS ACHIEVED

- On SPEC CPU 2006, Hawkeye reduces LLC miss rate over LRU by 17.0% and improves IPC by 8.4%, versus 11.4%/6.2% for SDBP and 11.7%/5.6% for SHiP, the best prior policies.
- On a 4-core, 8MB shared LLC, Hawkeye improves speedup over LRU by 15.0%, versus 12.1% for SDBP and 11.4% for SHiP; OPTgen reconstructs OPT with 99% accuracy, and the Hawkeye Predictor itself is 81% accurate, all within a 28KB hardware budget.

IV. KEY TAKEAWAY

- 1) An infeasible optimal algorithm can still be useful indirectly, not by running it, but by simulating what it would have done on data already observed and learning that pattern; Belady's

algorithm cannot see the future, but its past verdicts are fully computable after the fact.

- 2) Reuse distance alone is an incomplete signal; what matters is demand relative to capacity at the time, which is why liveness intervals, tracking how many lines compete for the cache simultaneously, succeed where distance-only predictors fail.
- 3) Correlating a decision with the instruction that caused it, rather than with the address or content of the data itself, generalizes better because instructions are far fewer in number and their behavior across invocations is more stable than the specific memory they happen to touch.

V. WEAKNESS

- The Hawkeye Predictor is only 81% accurate against OPTgen's 99%-accurate reconstruction, and the paper attributes some of this gap to slow or incorrect learning, but tests a confidence-augmented predictor only briefly and drops it for insufficient gain relative to hardware cost, leaving the predictor itself as the main acknowledged source of remaining loss.
- Multi-core evaluation trains one occupancy vector and one predictor on the interleaved access stream of all cores; the paper's own scheduling-effects discussion admits that non-deterministic interleaving means the past optimal solution may not represent future optimal decisions, and only reports the resulting OPT bias, 89%, without evaluating a per-core alternative.
- OPTgen's accuracy and the Hawkeye Predictor's accuracy are evaluated separately, but the paper does not report how errors in one stage compound with errors in the other under adversarial phase-change conditions, where the LRU fallback and detraining logic must both correct for approximation simultaneously.

VI. EXTENSION

- The paper's own conclusion names using more sophisticated predictors to learn the OPTgen oracle more precisely, since OPTgen already reaches 99% accuracy while the predictor reaches only 81%, as the direction with the greatest remaining potential.
- The Hawkeye Predictor uses only PC identity to index its table; an untried alternative is combining PC with a short history of the line's own recent hit/miss outcomes, giving the predictor a per-line correction signal on top of the per-PC bias without abandoning the low-cost table lookup.
- OPTgen's occupancy vector already encodes cache demand as a byproduct of computing OPT; this demand signal is discarded after training the predictor, but exposing it directly, for example to bias insertion more aggressively during high-demand phases detected by the vector itself, is unexplored and would reuse state already being computed.

VII. REFERENCE

Jain and Lin, *Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement*, ISCA 2016.